Task 20 — End-to-End Pipelines & Deployment (Pickle/Flask)

AI / ML Developer · Phase 1 Industry Immersion · Task Brief

AI / ML DEVELOPER PlaceMux · Phase 1 · Task Brief

DIFFICULTY **Mastery · Expert** Level 20 of 20 · ramp 95%

Objective

What this task is: Build the capstone: an end-to-end pipeline deployed behind a simple app/API (e.g. Pickle + Flask) for live predictions.

Why it matters: This proves the full arc -- data to deployed, callable model -- the core competency Phase 1 is building toward.

What you will deliver

- A deployed model service with a validated endpoint, health/error handling, and a live end-to-end demo.

Inputs & prerequisites

- A reproducible ML environment with fixed seeds.
- Train/validation/test discipline and a baseline-first mindset.
- Honest evaluation: the right metric, on unseen data.
- Required tools for this task: Flask/FastAPI, joblib, pydantic/validation, curl/Postman, Gradio/Streamlit (optional UI)

Step-by-step execution

Work through these in order. Don't skip the verification steps — that's where 'looks done' becomes 'is done'.

1. Load the serialised pipeline in a small Flask/FastAPI service.
2. Define a validated prediction endpoint (input -> score).
3. Add a health check and graceful error handling.
4. Test with real-shaped requests end-to-end.
5. Measure latency and confirm it's acceptable.
6. Prepare a live demo: send a request, get a sensible prediction.

Tools & stack

- Flask/FastAPI
- joblib
- pydantic/validation
- curl/Postman
- Gradio/Streamlit (optional UI)

Definition of Done

You are done when all of these are true and demoable

- A deployed model service with a validated endpoint, health/error handling, and a live end-to-end demo.
- Demonstrable live on real (even if small) data, not just described.

Pitfalls to avoid

Worry if any of these are true of your work

- No input validation on the endpoint.
- Ignoring latency.
- Can't reproduce the served model.

Where this fits in the track

Over Phase 1 you go from a clean environment to a deployed, callable model. You learn to build models you can measure, explain and serve -- not just ones that run.